Name:Mayuri Hirade

Roll no:13163

In [40]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import confusion_matrix, accuracy_score, precision_score, recall_score
```

In [41]:
```
iris = pd.read_csv(r'C:\Users\AKSHATA\Downloads\Iris.csv')
```

In [42]:
```
iris
```

Out[42]:

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	1	5.1	3.5	1.4	0.2	Iris-setosa
1	2	4.9	3.0	1.4	0.2	Iris-setosa
2	3	4.7	3.2	1.3	0.2	Iris-setosa
3	4	4.6	3.1	1.5	0.2	Iris-setosa
4	5	5.0	3.6	1.4	0.2	Iris-setosa
...	...	...	...	...	...	...
145	146	6.7	3.0	5.2	2.3	Iris-virginica
146	147	6.3	2.5	5.0	1.9	Iris-virginica
147	148	6.5	3.0	5.2	2.0	Iris-virginica
148	149	6.2	3.4	5.4	2.3	Iris-virginica
149	150	5.9	3.0	5.1	1.8	Iris-virginica

150 rows × 6 columns

In [43]:
```
iris.isnull()
```

Out[43]:

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	False	False	False	False	False	False
1	False	False	False	False	False	False
2	False	False	False	False	False	False
3	False	False	False	False	False	False
4	False	False	False	False	False	False
...	...	...	...	...	...	...
145	False	False	False	False	False	False
146	False	False	False	False	False	False
147	False	False	False	False	False	False
148	False	False	False	False	False	False
149	False	False	False	False	False	False

150 rows × 6 columns

In [44]:
```
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import f1_score
label_encoder = LabelEncoder()
iris["Species"] = label_encoder.fit_transform(iris["Species"])
```

In [45]:
```
X = iris.iloc[:, :-1]
y = iris.iloc[:, -1]
```

In [46]:
```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

In [47]:
```
model=GaussianNB()
model.fit(X_train,y_train)
```

Out[47]:

▾ GaussianNB

GaussianNB()

In [48]:
```
y_pred = model.predict(X_test)
```

In [49]:
```
# Compute Confusion Matrix
```

```
cm = confusion_matrix(y_test, y_pred)
tp = cm[0, 0]  # True Positives
tn = cm[1, 1]  # True Negatives
fp = cm[1, 0]  # False Positives
fn = cm[0, 1]  # False Negatives
```

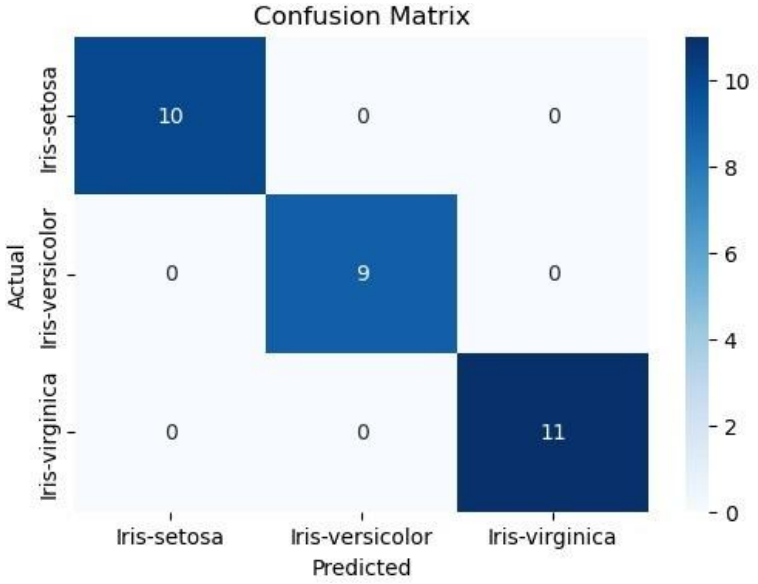
In [50]:
```
# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
TN, FP, FN, TP = cm.ravel()[:4]  # Extract values
```

In [51]:
```
# Performance metrics
accuracy = accuracy_score(y_test, y_pred)
error_rate = 1 - accuracy
precision = precision_score(y_test, y_pred, average="macro")
recall = recall_score(y_test, y_pred, average="macro")
f1 = f1_score(y_test, y_pred, average="macro")
```

In [52]:
```
print(f"Confusion Matrix:\n{cm}")
print(f"True Positives (TP): {TP}")
print(f"False Positives (FP): {FP}")
print(f"True Negatives (TN): {TN}")
print(f"False Negatives (FN): {FN}")
print(f"Accuracy: {accuracy:.2f}")
print(f"Error Rate: {error_rate:.2f}")
print(f"Precision: {precision:.2f}")
print(f"Recall: {recall:.2f}")
print(f"F1 Score: {f1:.2f}")
```

```
Confusion Matrix:
[[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]
True Positives (TP): 0
False Positives (FP): 0
True Negatives (TN): 10
False Negatives (FN): 0
Accuracy: 1.00
Error Rate: 0.00
Precision: 1.00
Recall: 1.00
F1 Score: 1.00
```

In [53]:
```
plt.figure(figsize=(6, 4))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", xticklabels=label_encoder.classes_, yticklabels=label_encode
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()
```



In [ ]:
```
```